

주가 데이터와 경제 지표를 통합한 이중 입력 모델 설계

1. 라이브러리 импорт (Import)

```
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Model
from tensorflow.keras.layers import (
    Input, Dense, Dropout, LayerNormalization, MultiHeadAttention, Add,
    GlobalAveragePooling1D
)
import tensorflow as tf
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
```

1. **google.colab.drive**

- `**drive.mount('/content/drive')**`를 통해 Google Drive와 연동합니다.
- Colab에서 Drive 내부 파일을 읽고 쓸 수 있게 해줍니다.

2. **pandas (pd)**

- 데이터 분석과 처리에 필수적인 라이브러리.

- CSV 파일을 읽고, DataFrame으로 관리할 수 있습니다.

3. **numpy (np)**

- 수학적 연산과 배열 처리를 간단하게 해주는 라이브러리.

4. **sklearn.preprocessing.MinMaxScaler**

- 데이터를 0과 1 사이로 스케일링해주는 도구.
- 신경망 학습에서 특정 범위(예: 0~1)로 값들을 맞추면 학습이 더 안정적으로 진행됩니다.

5. **tensorflow.keras**

- **딥러닝 모델**을 만드는 핵심 라이브러리.
- 이 중 **Model, Input, Dense, Dropout, MultiHeadAttention** 등을 가져와서 Transformer 구조를 구현합니다.

6. **matplotlib (plt) & matplotlib.dates (mdates)**

- 그래프(Plot) 그리기에 사용.
- 시계열(Date) 데이터를 시각화하기 위한 기능 포함.

2. Transformer 인코더 함수 정의

```
def transformer_encoder(inputs, num_heads, ff_dim, dropout=0.1):  
    attention_output = MultiHeadAttention(num_heads=num_heads,  
key_dim=inputs.shape[-1])(inputs, inputs)  
    attention_output = Dropout(dropout)(attention_output)  
    attention_output = Add()([inputs, attention_output])  
    attention_output = LayerNormalization(epsilon=1e-6)(attention_output)  
  
    ffn = Dense(ff_dim, activation="relu")(attention_output)  
    ffn = Dense(inputs.shape[-1])(ffn)
```

```
ffn_output = Dropout(dropout)(ffn)

ffn_output = Add()([attention_output, ffn_output])

ffn_output = LayerNormalization(epsilon=1e-6)(ffn_output)

return ffn_output
```

1. MultiHeadAttention

- 여러 개의 Attention Head가 각 시점 정보를 병렬로 학습합니다.
- **inputs** 자체가 Query, Key, Value 모두로 쓰이므로, **Self-Attention** 구조입니다.

2. Add & LayerNormalization

- **Add()**: Residual Connection(잔차 연결).
- **LayerNormalization**: 각 시점마다 Feature 방향으로 정규화해, 학습을 안정화합니다.

3. Feed-Forward Network (FFN)

- **Dense(ff_dim, activation="relu")** → **Dense(inputs.shape[-1])** 구조로,
- Attention 출력을 추가적으로 가공합니다.

4. Dropout

- Overfitting을 방지하기 위해 일부 뉴런을 확률적으로 0으로 만듭니다.

정리: 이 함수는 **한 층의 Transformer 인코더**를 구현합니다. 입력을 받고 Self-Attention + FeedForward 과정을 거쳐 다시 반환합니다.

3. Transformer 모델 생성 함수

```
def build_transformer_with_two_inputs(stock_shape, econ_shape, num_heads, ff_dim,
target_size):
```

...

여기서 ****주가 데이터(Stock)와 경제 지표(Econ)****라는 두 가지 입력을 동시에 처리합니다.

1. 두 개의 Input Layer

- `stock_inputs = Input(shape=stock_shape)`
- `econ_inputs = Input(shape=econ_shape)`
- 각각 시계열 길이(**lookback**), Feature 수가 달라질 수 있습니다.

2. Transformer 인코더 반복

- 주식 입력에 대해 4번(**for _ in range(4)**) Transformer 레이어를 쌓습니다.
- 경제 지표 입력에 대해서도 동일하게 4번 레이어를 쌓습니다.

3. Dense(64, activation="relu")

- 인코딩된 출력을 64차원으로 변환해줍니다.

4. 병합(Add) 및 추가 Dense

- ****merged = Add()([stock_encoded, econ_encoded])****로 두 인코딩 결과를 합칩니다.
- ****merged = Dense(128, activation="relu")(merged)****로 통합된 정보를 한 번 더 가공합니다.

5. Dropout + GlobalAveragePooling1D

- 시퀀스 차원을 평균으로 pooling하여 최종적으로 **(Batch, Feature)** 형태로 만듭니다.
- **outputs = Dense(target_size)(merged)**는 최종적으로 예측해야 할 출력 차원(**target_size**)을 지정합니다.

6. 모델 생성

7.

8. `return Model(inputs=[stock_inputs, econ_inputs], outputs=outputs)`

9.

- 2개 Input, 1개 Output 구조를 갖는 Keras 모델을 반환합니다.

4. 데이터 로딩 & 전처리

```
file_path = '/content/drive/My Drive/total.csv'
```

```
data = pd.read_csv(file_path, parse_dates=['날짜'])
```

```
data.sort_values(by='날짜', inplace=True)
```

- **CSV 파일 로드:** Google Drive에 저장된 **total.csv** 파일을 Pandas DataFrame으로 읽어옵니다.
- ****parse_dates=['날짜']****를 통해 날짜 컬럼을 실제 날짜 타입으로 변환.
- 날짜 기준으로 정렬(**sort_values**)해 시계열 순서를 맞춥니다.

```
data.fillna(method='ffill', inplace=True)
```

```
data.fillna(method='bfill', inplace=True)
```

```
data = data.apply(pd.to_numeric, errors='coerce')
```

```
data.dropna(inplace=True)
```

- **결측치 처리:** **ffill**, **bfill**(앞뒤 값으로 채우기)을 사용해 NaN을 최대한 보완 후, 그래도 남아있는 NaN은 제거(**dropna**)합니다.

```
forecast_horizon = 7
```

- **일주일 후를 예측**한다는 뜻입니다. 예: 오늘 데이터(오늘 주가 & 경제 지표)를 보고 7일 뒤 주가를 예측.
-

5. 컬럼 설정

```
target_columns = [  
    'QQQ ETF'  
]
```

- 예측 대상(타겟)은 **QQQ ETF**만 사용하고 있습니다.
- 물론 여러 종목(애플, MS 등)을 예측하려면 리스트를 더 채우면 됩니다.

```
economic_features = [  
    '10년 기대 인플레이션율', '장단기 금리차', '기준금리', ...  
    # 다양한 경제 지표 'VIX 지수'  
]
```

- 시계열 모델에 투입할 **경제 지표**들입니다.

6. 데이터 스케일링

```
train_size = int(len(data) * 0.8)  
train_data = data.iloc[:train_size]  
test_data = data.iloc[train_size:]
```

- 데이터 중 80%는 훈련, 20%는 테스트로 구분합니다.

```
stock_scaler = MinMaxScaler()
```

```
econ_scaler = MinMaxScaler()
```

```
data_scaled[target_columns] = stock_scaler.fit_transform(data[target_columns])
```

```
data_scaled[economic_features] = econ_scaler.fit_transform(data[economic_features])
```

- **MinMaxScaler**: 각 컬럼 값을 0~1 범위로 변환.
- **stock_scaler**: 주가(타겟 컬럼)에 사용.
- **econ_scaler**: 경제 지표에 사용.
- 이렇게 분리해서 사용하면 나중에 역변환(inverse_transform)도 편리해집니다.

7. 시계열 학습용 데이터셋 구성

```
lookback = 90
```

- 90일(약 3개월)의 과거 데이터를 보고 **미래**를 예측하도록 합니다.

```
X_stock_train, X_econ_train, y_train = [], [], []
```

```
for i in range(lookback, len(data_scaled) - forecast_horizon):
```

```
    X_stock_seq = data_scaled[target_columns].iloc[i - lookback:i].to_numpy()
```

```
    X_econ_seq = data_scaled[economic_features].iloc[i - lookback:i].to_numpy()
```

```
    y_val = data_scaled[target_columns].iloc[i + forecast_horizon - 1].to_numpy()
```

```
    X_stock_train.append(X_stock_seq)
```

```
    X_econ_train.append(X_econ_seq)
```

```
y_train.append(y_val)
```

- 슬라이딩 윈도우 기법:

1. ***i***번째를 예측하려면, ***i*-90 ~ *i*-1**까지 데이터를 입력으로 사용.
2. 예측해야 할 값은 ***i* + (forecast_horizon - 1)** 즉 7일 후의 값.

- 결과적으로:

- ***X_stock_train***은 (샘플 수, 90, 타겟 컬럼 수) 형태.
- ***X_econ_train***은 (샘플 수, 90, 경제 지표 수).
- ***y_train***은 (샘플 수, 타겟 컬럼 수).

```
X_stock_train = np.array(X_stock_train)
```

```
X_econ_train = np.array(X_econ_train)
```

```
y_train = np.array(y_train)
```

- 리스트를 numpy 배열로 변환. 딥러닝 모델에서 배치 처리를 위해 필요합니다.

8. 전체 예측용 데이터 생성

```
X_stock_full, X_econ_full = [], []
```

```
for i in range(lookback, len(data_scaled)):
```

```
    X_stock_seq = data_scaled[target_columns].iloc[i - lookback:i].to_numpy()
```

```
    X_econ_seq = data_scaled[economic_features].iloc[i - lookback:i].to_numpy()
```

```
    X_stock_full.append(X_stock_seq)
```

```
    X_econ_full.append(X_econ_seq)
```

- 여기는 **훈련이 끝난 후, 최종적으로 전체 기간을 예측하는 용도**.
- ****forecast_horizon****을 빼지 않고, 마지막 데이터까지 포괄적으로 입력 시퀀스를 준비해둡니다.

```
X_stock_full = np.array(X_stock_full)
```

```
X_econ_full = np.array(X_econ_full)
```

- 마찬가지로 numpy 배열로 변환.

9. Transformer 모델 빌드 & 학습

```
stock_shape = (lookback, len(target_columns))
```

```
econ_shape = (lookback, len(economic_features))
```

```
model = build_transformer_with_two_inputs(
```

```
    stock_shape, econ_shape,
```

```
    num_heads=8, ff_dim=256,
```

```
    target_size=len(target_columns)
```

```
)
```

```
model.compile(optimizer=Adam(learning_rate=0.0001), loss='mse', metrics=['mae'])
```

```
model.summary()
```

- **stock_shape**: (90, 1) → 90일, 1개 종목(QQQ).
- **econ_shape**: (90, 경제 지표 수).
- **num_heads=8, ff_dim=256**: Transformer의 어텐션 헤드 수, FeedForward 차원.

- **loss='mse'**: 평균제곱오차 사용.
- **metrics=['mae']**: 평균절대오차를 함께 모니터링.

```
history = model.fit([X_stock_train, X_econ_train], y_train, epochs=50, batch_size=32,
verbose=1)
```

- 실제 학습 단계.
- **epochs=50**번 반복, **batch_size=32**씩 처리.

10. 전체 예측

```
predicted_prices = model.predict([X_stock_full, X_econ_full], verbose=1)
predicted_prices_actual = stock_scaler.inverse_transform(predicted_prices)
```

- 학습된 모델을 사용해 **X_stock_full**, **X_econ_full**에 대한 예측값을 얻습니다.
- **inverse_transform**: 0~1 사이로 스케일링했던 값을 원상 복구(실제 주가 단위)합니다.

11. 예측 결과 정리

```
pred_len = len(predicted_prices_actual)
today_dates = data['날짜'].iloc[lookback : lookback + pred_len].values
```

- 예측된 샘플 수(**pred_len**)에 맞춰 날짜 인덱스를 준비.

```
actual_data_end = min(lookback + pred_len, len(data))
```

```
actual_full = data[target_columns].iloc[lookback:actual_data_end].values
```

- 실제값(**Actual**)도 동일 구간으로 가져옵니다.

```
if actual_full.shape[0] < pred_len:
```

```
    nan_padding = np.full((pred_len - actual_full.shape[0], len(target_columns)), np.nan)
```

```
    actual_full = np.vstack([actual_full, nan_padding])
```

- 실제값이 예측 샘플보다 적으면, 남은 구간은 NaN으로 채워둡니다.

12. 결과 DataFrame에 담기

```
result_data = pd.DataFrame({'날짜': today_dates})
```

```
for idx, col in enumerate(target_columns):
```

```
    result_data[f'{col}_Predicted'] = predicted_prices_actual[:, idx]
```

```
    result_data[f'{col}_Actual'] = actual_full[:, idx]
```

- 예: **QQQ ETF_Predicted**, **QQQ ETF_Actual** 열을 생성.

13. CSV로 저장 & 시각화

```
output_file_path = '/content/drive/My Drive/predicted_stock.csv'
```

```
result_data.to_csv(output_file_path, index=False)
```

```
print(f"Predicted stock returns saved to {output_file_path}")
```

- 예측 결과를 CSV로 저장, Drive에 업로드합니다.

(1) 학습 손실 시각화

```
plt.figure(figsize=(12, 6))

plt.plot(history.history['loss'], label='Train Loss')

plt.title('Training Loss')

plt.xlabel('Epoch')

plt.ylabel('Loss')

plt.legend()

plt.show()
```

- 에폭(Epoch)마다의 손실이 어떻게 변하는지 확인.

(2) 실제값 vs. 예측값 (7일 후)

```
for col in target_columns:

    plt.figure(figsize=(12, 6))

    plt.plot(pd.to_datetime(result_data['날짜']), result_data[f'{col}_Actual'], label='Actual
(Today)', alpha=0.7)

    plt.plot(pd.to_datetime(result_data['날짜']), result_data[f'{col}_Predicted'],
label='Predicted (7 days later)', alpha=0.7)

    ...

    plt.show()
```

- **x축**은 "오늘 날짜"
- *****_Actual*****은 오늘 실제 주가, ******_Predicted******은 7일 뒤 예측 주가를 오늘 날

짜에 그려 시각적 비교.

정리 및 핵심 포인트

1. 데이터 전처리

- 결측치 처리 & 스케일링이 제대로 되어야 모델이 안정적으로 학습.

2. Transformer 구조

- 주가 입력(Stock) + 경제 지표 입력(Econ) → 병합.
- Attention으로 장기 패턴 & 많은 Feature 간 상호작용을 학습할 수 있음.

3. 슬라이딩 윈도우 방식

- **lookback** 기간의 데이터를 보고, ****forecast_horizon****일 뒤를 예측.
- QQQ ETF만 예측했지만, 다른 타겟도 추가 가능.

4. 예측 결과 해석

- "Actual (Today)" vs. "Predicted (7 days later)" 곡선을 비교해 **정도**를 평가.
- 별도 테스트셋을 구성하거나, 평가 지표(예: MSE, RMSE, MAE)를 추가하면 정확도를 수치로 확인할 수 있음.

이상으로, **시계열 Transformer 모델**로 주가를 예측하는 전체 흐름이었습니다. 실제로 모델이 잘 일반화되려면 **하이퍼파라미터 튜닝**이나, **데이터 구간 조절**이 필요할 수 있다는 점도 기억해두세요.

추가 팁

- **과적합(Overfitting) 체크**: 에폭 수를 늘려보거나, 검증셋(Validation set)을 활용.
- **다양한 경제 지표**를 시도해보거나, 특정 구간(예: 최근 5년)만 학습해볼 수도 있습니다.
- **LSTM과 비교**: 동일 데이터로 LSTM 모델도 구현해보고 결과를 비교하면, 어떤 구조가 더 나은지 실제로 체험할 수 있습니다.